



Orientação a Objetos

Quais são as partes do meu software?



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Quais são as peças que usamos em software orientado a objetos ?



Cão



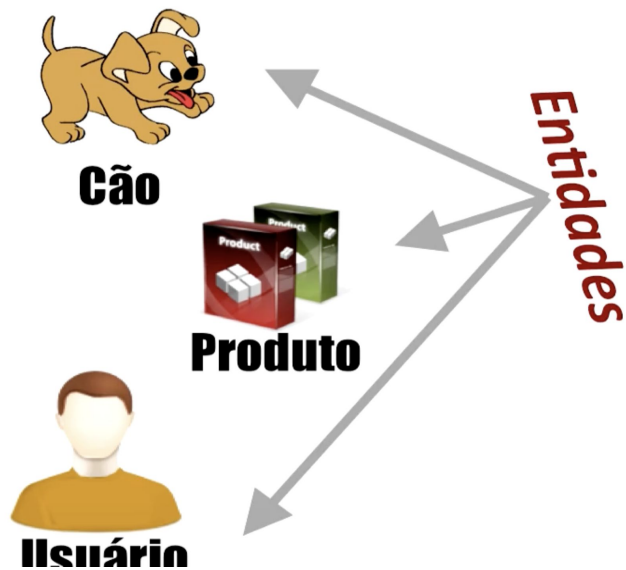
Produto



Usuário

Software de um pet-shop





Software de um pet-shop





Cão

*Entidades colaboram
para criar a
funcionalidade do
software*



Produto



Usuário

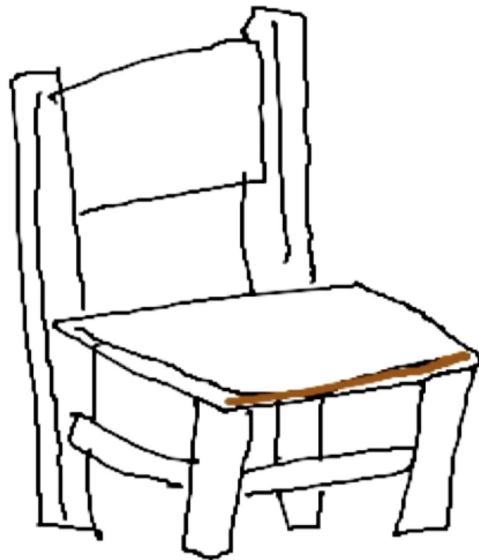
Software de um pet-shop



**As entidades que
colaboram na
funcionalidade de um
sistema são as suas
classes !**



A classe é uma abstração



Exemplares da **classe Cadeira**



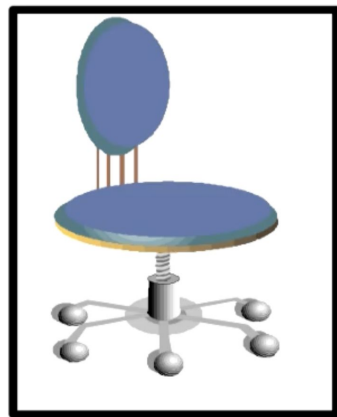
SOFTEX
PERNAMBUCO

 **Softex**

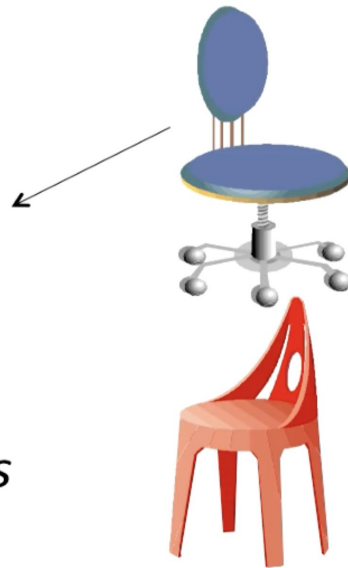
MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO





*características
próprias*



Instância Concreta = Objeto

Classe: Carro



Instância



Objeto: MeuCarro

Cor: []
Ano: []
Velocidade: km/h



Objeto: CarroFamilia

Cor: []
Ano: []
Velocidade: km/h



CarroTrabalho

Cor: []
Ano: []
Velocidade: km/h



BOLSA
FUTURO
DIGITAL



**Classe
Cadeira**

**cria /
Instancia**

**Objetos
Cadeira**

Objetos são únicos!



a)



b)



c)



**Qual dos
objetos são
instância da
classe
Cadeira?**



a)

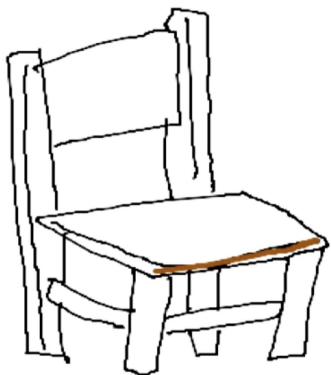


b)



c)





Abstrato

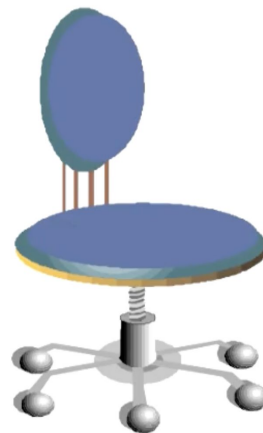




Abstrato

Concreto





Abstrato

Concreto

Concreto



Duas cadeiras visualmente iguais podem ser objetos diferentes ?



Abstrato

Concreto

Concreto



Duas cadeiras visualmente iguais podem ser objetos diferentes ? Sim !



Abstrato

Concreto

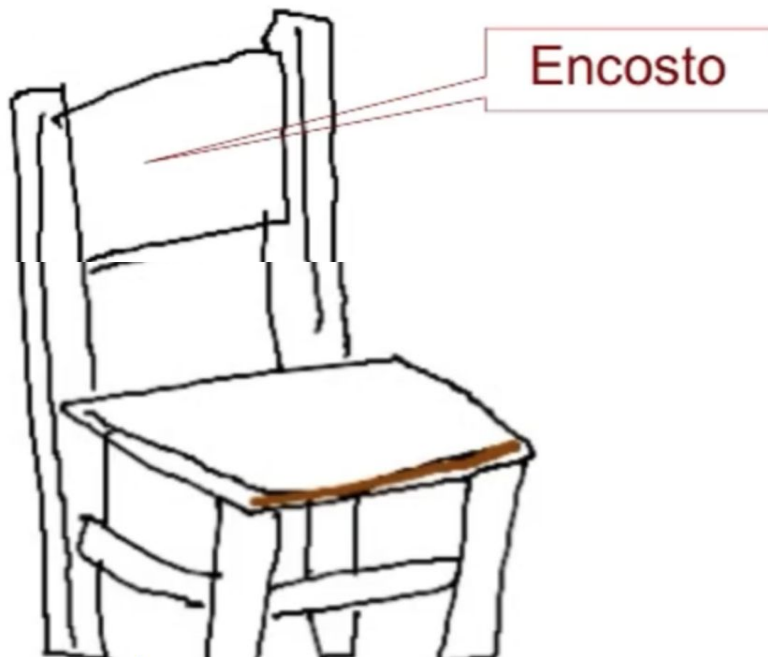
Concreto



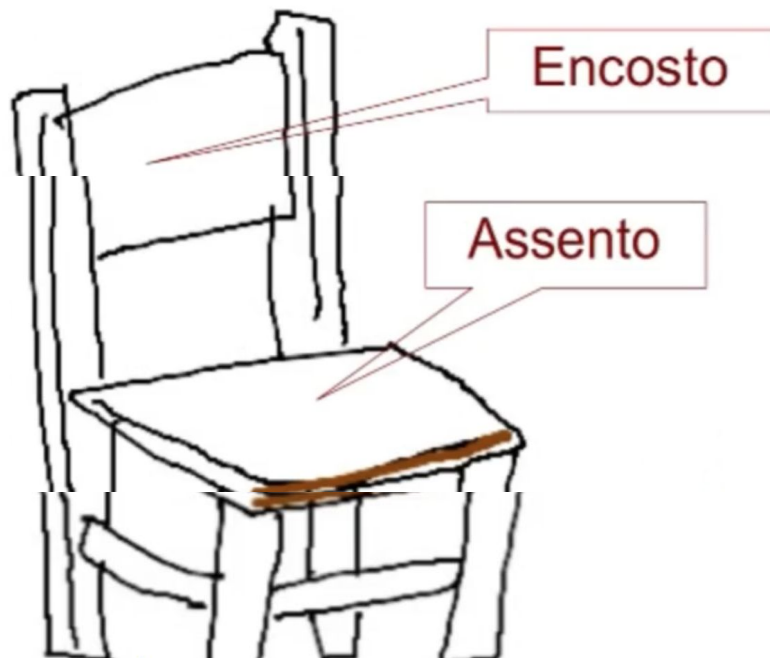
Comportamento e Estados das Classes



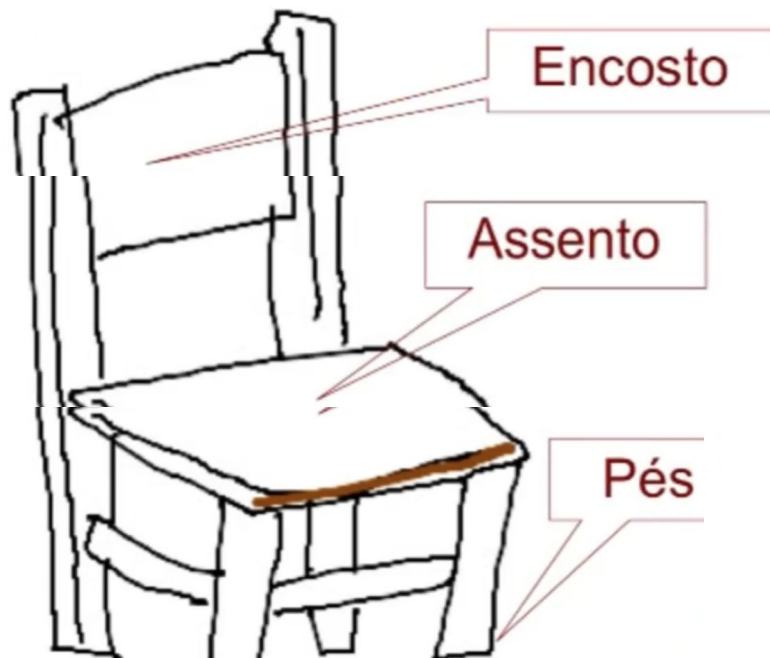
Comportamento e Estados das Classes



Comportamento e Estados das Classes



Comportamento e Estados das Classes



Comportamento e Estados das Classes

**Cada objeto da classe
Cadeira é diferente!**



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



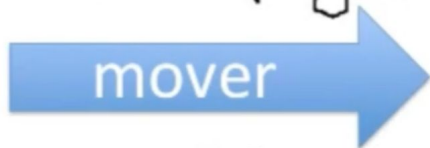
Comportamento e Estados das Classes

**As características de
um objeto da classe
são chamadas de
atributos!**



O estado de uma classe é representado pelos Atributos

Comportamento e Estados das Classes



**A classe também
pode realizar ações!**

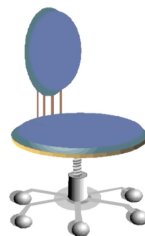
Comportamento e Estados das Classes

**Uma ação pode inclusive
mudar valores de
atributos de um objeto!**



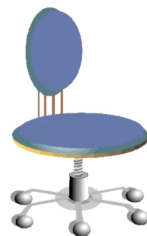
Comportamento e Estados das Classes

**Uma ação pode inclusive
mudar valores de
atributos de um objeto!**



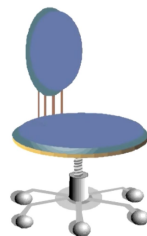
Comportamento e Estados das Classes

**Uma ação pode inclusive
mudar valores de
atributos de um objeto!**



Comportamento e Estados das Classes

**Uma ação pode inclusive
mudar valores de
atributos de um objeto!**



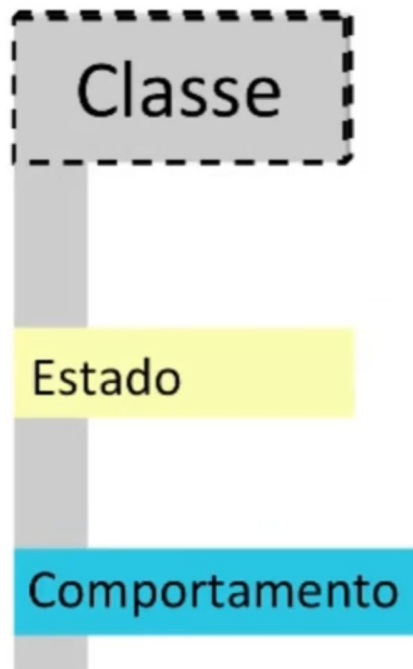
Comportamento e Estados das Classes

**As ações que um
objeto da classe
pode realizar são
representadas por
métodos!**

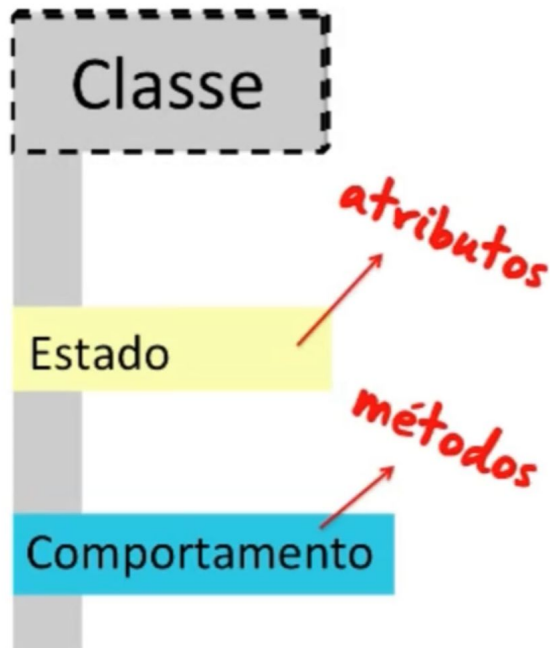


O comportamento de uma classe é representado pelos Métodos

Comportamento e Estados das Classes



Comportamento e Estados das Classes



Comportamento e Estados das Classes



Classe
Carro

Potência Motor
Velocidade

Acelerar
Frear



Comportamento e Estados das Classes



Objeto
Carro do
Meu Sogro

Potência Motor = 2.0
Velocidade = 60 km/h

Acelerar
Frear



Comportamento e Estados das Classes

Classe	Objeto
Abstrato	Concreto
Tem atributos	Tem valores
Tem ações	Executa ações



Criando classes com TypeScript



*A informação que
uma classe precisa
saber é armazenada
nos atributos*

```
class Carro {  
  potencia?: number  
  velocidade?: number  
}
```



→
*Cada objeto da classe
terá um valor diferente.*



Criando classes com TypeScript

**Todo atributo
precisa ter um
tipo!**

enginePower: number

Um atributo pode ser
de um tipo primitivo
ou de uma classe!



Criando classes com TypeScript



Criando Objetos

Usa o operador
new

`const carro = new Carro()`



Criando classes com TypeScript



```
carro.potencia = 2  
const v = carro.velocidade
```

Os atributos
podem ser
acessados para
escrita e leitura



Criando classes com TypeScript

Comportamento



*Os métodos definem
o comportamento
da classe.*

```
class Carro {  
  potencia: number  
  velocidade: number  
  
  acelerar(){  
    this.velocidade += this.potencia  
  }  
  frear(){  
    this.velocidade *= 0.5  
  }  
}
```



Criando classes com TypeScript



Executando métodos

```
fusca.acelerar();  
fusca.frear();
```

O método será
executado no
objeto utilizado
para a chamada



Exercício 1

Criar a classe `Carro`.

Tipo	Nome	Inicialização (Valores Padrão)
Atributo	<code>potencia: number</code>	Começa com <code>= 0;</code>
Atributo	<code>velocidade: number</code>	Começa com <code>= 0;</code>
Atributo	<code>nome: string</code>	Começa com <code>= "";</code> (String vazia)
Método	<code>acelerar()</code>	Aumenta o valor da <code>velocidade</code> .
Método	<code>frear()</code>	Diminui o valor da <code>velocidade</code> .
Método	<code>exibirVelocidadeAtual()</code>	Deve imprimir: <code>"0 carro [nome do carro] está a [x] km/h."</code>



Exercício 2

Etapa	Tarefa a Ser Executada
1. Instanciação	Crie um novo objeto (fusca) baseado na classe Carro .
2. Atribuição	Configure o fusca : Nome: "Fusca 1988" ; Potência: 2 .
3. Aceleração	Faça o carro ganhar velocidade, acelere 3 vezes.
4. Status	Exiba a velocidade atual no console



Exercício 3



Etapa	Tarefa a Ser Executada
1. Instanciação	Crie um segundo objeto a partir da classe <code>Carro</code> . Chame esta nova instância de <code>ferrari</code> .
2. Configuração	Atribua as características à <code>ferrari</code> : Nome: "Ferrari GTB" ; Potência: 15.
3. Movimento (Ferrari)	Faça a Ferrari ganhar velocidade apenas 1 vez. Em seguida, reduza sua velocidade 2 vezes.
4. Status (Ferrari)	Solicite que a Ferrari informe a sua condição atual de velocidade no console.
5. Comparação	Imprima uma frase no console mostrando qual carro está atualmente com a maior velocidade <code>ferrari</code> ou <code>fusca</code> .

Exercício 4

Implemente uma classe chamada Paciente que possui um construtor que recebe o seu peso em quilos e sua altura em metros, ambos utilizando o tipo number. Crie um método chamado calcularIMC() que calcula o índice de Massa Corporal de acordo com a fórmula $IMC = \text{peso (quilos)} / (\text{altura} * \text{altura (metros)})$. Crie também um método chamado diagnostico() que utiliza o método calcularIMC() e retorna uma String de acordo com as seguintes faixas de valores:

- **Baixo peso muito grave** = IMC abaixo de 16 kg/m^2
- **Baixo peso grave** = IMC entre 16 e $16,99 \text{ kg/m}^2$
- **Baixo peso** = IMC entre 17 e $18,49 \text{ kg/m}^2$
- **Peso normal** = IMC entre 18,50 e $24,99 \text{ kg/m}^2$
- **Sobrepeso** = IMC entre 25 e $29,99 \text{ kg/m}^2$
- **Obesidade grau I** = IMC entre 30 e $34,99 \text{ kg/m}^2$
- **Obesidade grau II** = IMC entre 35 e $39,99 \text{ kg/m}^2$
- **Obesidade grau III (obesidade mórbida)** = IMC igual ou maior que 40 kg/m^2

Crie 3 instâncias da classe Paciente com valores diferentes e depois exiba no console o diagnóstico dos 3 pacientes.